Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

ReConnect: AI/Data-Driven Network Optimization for Disaster Recovery

Student / Team Information

Team Name: *Team #*	*CODE-IN-MOLE* ***STAT-IV-T039***
Team member 1 (Team Lead)	*Maini, Gunottam – 8209891651* *gunottamaini@gmail.com* 
Team member 2	*Garg,Ananya – 9996979997* *23022127@geu.ac.in* 

Team member 3	*Sharma, Aryan* *aryan.sharma151205@gmail.com* 
Team member 4	*Rawat, Aditya* *adrwt1233@gmail.com* 

PROJECT PROGRESS DESCRIPTION

Project Abstract

Traditional bus systems rely on fixed routes, which can result in inefficiencies, such as empty buses operating on low-demand routes while overcrowding occurs on others. This situation not only wastes fuel and increases operational costs but also frustrates passengers. Furthermore, individuals in less populated areas often experience long wait times, which discourages the use of public transit.

Our Dynamic Bus Route Generator effectively addresses this issue by adapting routes in real-time based on passenger demand. It optimizes pathways to minimize idle trips, reduce emissions, and enhance convenience—an essential feature in rapidly growing cities where fixed routes cannot meet changing needs.
For operators, it enhances efficiency by minimizing unnecessary stops and reducing fuel consumption. For passengers, it offers faster and more responsive service. Most importantly, it promotes sustainable mobility by positioning public transit as a more intelligent alternative to private vehicles.

Using intelligent algorithms, we bridge the gap between static schedules and real-time demand, paving the way for a greener and more user-focused urban transportation system.
Current bus systems predominantly rely on static route planning that utilizes graph algorithms such as Dijkstra's algorithm or heuristic-based optimizations. Some advanced approaches include periodic re-optimization through genetic algorithms or integer linear programming (conducted offline), demand-responsive transit with dynamic stop insertion (limited to small-scale operations), and flex-route buses that allow minor real-time detours (constrained by predefined corridors). However, these solutions either lack real-time adaptability, struggle with large-scale implementation, or provide only partial flexibility. The primary gap remains the absence of an efficient, scalable algorithm capable of fully regenerating bus routes in real-time based on dynamic demand, underscoring the need for more sophisticated algorithmic solutions in public transportation systems.

Project Goals

This project aims to revolutionize urban public transportation by developing a dynamic bus route generator [1][7]. Our primary objectives are threefold: First, we seek to create a real-time adaptive routing system that responds instantly to fluctuations in passenger demand [5]. Second, we aim to optimize resource allocation by minimizing operational costs while maximizing service coverage [3]. Third, we intend to design a scalable solution that maintains efficiency even in large metropolitan networks with thousands of daily riders [4].

Key Features

The system will integrate intelligent algorithms [1][3] to process live stop requests and dynamically adjust routes. We will prioritize passenger convenience by minimizing wait times and avoiding unnecessary detours, while ensuring that the solution remains computationally feasible for city-wide implementation [4][7]. This technology will serve as a practical compromise between rigid fixed-route systems and costly on-demand transit options.

Updated Project Approach and Architecture

Our project utilizes a geospatial AI-driven approach to identify and prioritize critical infrastructure in Kathmandu, Nepal, for effective disaster response and planning. The system integrates multiple open-source geospatial datasets and uses statistical and machine learning techniques to estimate infrastructure vulnerability and importance.

System Design:
The project pipeline consists of four major stages:

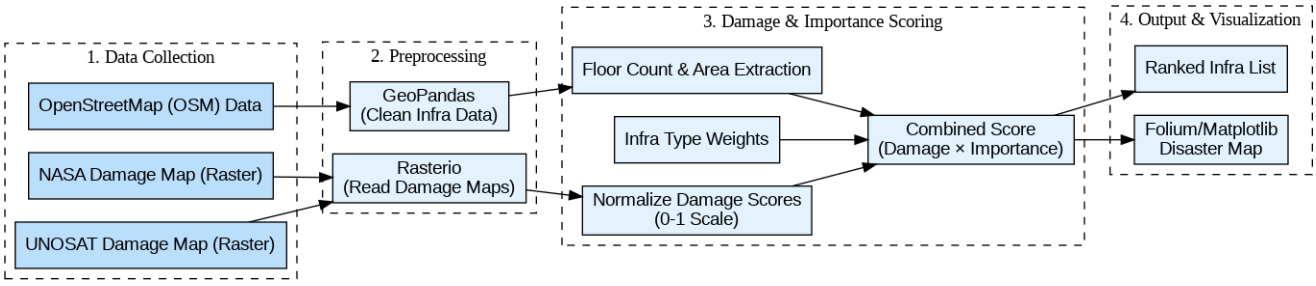
1. **Data Collection**: Geospatial data is gathered from OpenStreetMap (OSM) using the `osmnx` library, and satellite-derived damage maps are sourced from NASA and UNOSAT.
2. **Preprocessing**: The `GeoPandas` library is used to handle and manipulate geographic dataframes (GeoDataFrames). Missing values are handled using imputation techniques (`fillna`), and features like building area and number of floors are cleaned and standardized.
3. **Damage Estimation**: Satellite raster damage data (GeoTIFF format) is read using `rasterio`. The pixel values are normalized and converted into a quantitative "damage score" for each infrastructure footprint.
4. **Scoring and Prioritization**: An "importance score" is calculated for each building based on its type (e.g., hospital, school), number of floors, area, and damage level. These scores are then used to rank infrastructures for response planning.

Communication Protocols:
Data flow within the system is file-based and procedural. Raster and vector data are read from disk and processed using Python libraries, without the need for network APIs during runtime.

Libraries Used:

- `osmnx` and `GeoPandas` for spatial feature extraction
- `rasterio` for reading satellite damage rasters
- `numpy` and `pandas` for data manipulation
- `matplotlib` and `folium` for visualization and mapping

This modular design ensures scalability, allowing future integration of real-time data feeds or optimization algorithms for emergency routing.



Tasks Completed

Task Completed	Team Member
Data Acquisition: Spatial data for buildings and infrastructure in Kathmandu was collected from OpenStreetMap using osmnx and geopandas. Raster damage assessment data from NASA and UNOSAT was also obtained to incorporate satellite-based damage insights.	Aditya Rawat, Aryan Sharma

Data Preprocessing: OSM data was filtered to focus on key infrastructure types like hospitals and schools. NASA and UNOSAT raster files were read using rasterio, and missing or invalid data points were carefully handled to ensure accuracy.	Aditya Rawat, Aryan Sharma
Damage Scoring: Damage values from NASA and UNOSAT raster data were normalized to a 0–1 scale for consistency. These normalized scores were spatially mapped to infrastructure points, integrating satellite damage data with ground infrastructure.	Gunottam Maini
Importance Weighting: Each infrastructure type was assigned an importance weight reflecting its criticality (e.g., hospitals = 1.0, shops = 0.3). Building features such as number of floors and area were also factored in to refine importance scores.	Gunottam Maini, Ananya Garg
Scoring & Ranking: A composite score for each infrastructure was computed by multiplying damage scores with importance weights. Based on this composite score, infrastructures were ranked to identify priority assets for disaster response.	Gunottam Maini, Ananya Garg
Visualization: Interactive maps were created using folium and matplotlib, highlighting high-priority structures. Color coding was applied to severity levels, enabling intuitive visualization of areas needing urgent attention.	Ananya Garg
Automated Infrastructure Prioritization Dashboard: We plan to develop a simple dashboard interface that will allow users (e.g., local authorities) to view, filter, and export prioritized infrastructure lists based on severity, type, and region.	Aryan Sharma
Validation with Ground Truth or Historical Events: The current prioritization is model-based. We intend to validate the results by comparing them with known disaster reports or historical damage data, if available, to assess the accuracy and usefulness of our scoring system.	Aditya Rawat
Multi-layer Damage Fusion: While NASA and UNOSAT damage rasters have been integrated individually, a unified fusion method combining both sources (e.g., averaging or max logic) for better reliability is yet to be implemented.	Ananya Garg

Weight Optimization using AI Techniques: At present, weights for infrastructure importance are manually assigned. We plan to use data-driven techniques, possibly simple machine learning models or heuristic optimization, to auto-tune these weights based on external disaster impact datasets.	Gunottam Maini
Performance Tuning & Scalability: With an increase in area coverage or more datasets, the current implementation may slow down. We plan to add spatial indexing (e.g., using rtree) and multiprocessing to improve performance.	Aryan Sharma & Aditya Rawat
Comprehensive Documentation & Code Packaging: Final documentation, README files, and modular code packaging for open-source sharing are pending and will be completed as the final deliverable.	Aryan Sharma & Aditya Rawawt

Challenges/Roadblocks

1. **CRS Mismatch & Area Calculation Errors:**
   One major issue was that our infrastructure geometries were initially in a geographic Coordinate Reference System (CRS), which caused incorrect area calculations. This resulted in inaccurate plinth area estimations. We resolved it by reprojecting the geometries to a projected CRS using `.to_crs(epsg=32645)` for Kathmandu.
2. **Handling Missing or Incomplete Data:**
   Many infrastructure entries from OSM were missing values for key fields such as `building:levels` or `area`. To address this, we used default values and `pandas.to_numeric(errors='coerce')` to handle invalid entries gracefully, replacing missing data with reasonable assumptions without distorting the overall analysis.
3. **Damage Raster Alignment with Vector Data:**
   Integrating raster data (NASA/UNOSAT) with vector infrastructure data was challenging due to differences in spatial resolution and alignment. We used spatial joins and masking techniques in `rasterio` and `shapely` to associate each building with its respective raster-based damage score.
4. **Weight Assignment Complexity:**
   Assigning objective and meaningful weights to different infrastructure types (e.g., hospitals vs. malls) was

initially subjective. We reviewed disaster management literature and government guidelines to define a justified and scalable weight distribution system.

5. **Visualization Overload on Maps:**
   Displaying many infrastructure points on a single map led to visual clutter. We solved this by filtering top-priority structures and using Folium's layer control and clustering features for better interaction.
6. **Computational Bottlenecks:**
   Processing large geospatial datasets caused performance issues. We optimized the code by reducing unnecessary computations and reading data selectively when required.

Tasks Pending

Task Pending	Team Member (to complete the task)
• **Deployment:** The key pending tasks for this project include deploying the Streamlit application on a cloud platform to make it accessible from any device, especially for emergency responders in the field. Additionally, authentication and basic access control need to be implemented to ensure secure interaction with the system. The UI should be further optimized for mobile responsiveness to support use on smartphones and tablets. For future enhancements, offline support and integration with real GIS layers from government or UNOSAT sources are planned, which will enable the application to work reliably in low-connectivity disaster zones and with actual infrastructure data.	Gunottam Maini, Aryan Sharma, Ananya Garg, Aditya Rawat

Project Outcome/Deliverables

The key outcome of our project is a data-driven decision-support tool to assist in disaster response planning for Kathmandu. The system integrates geospatial building and infrastructure data with satellite-based damage assessments to prioritize critical infrastructure for relief and recovery.

Deliverables include:

1. **Integrated Geospatial Dataset:**
   A cleaned and enriched geospatial dataset of infrastructure (hospitals, schools, government buildings, etc.) in Kathmandu, annotated with damage scores from NASA and UNOSAT raster data.
2. **Infrastructure Prioritization Model:**
   A scoring algorithm that ranks each infrastructure point based on combined damage severity and functional importance, accounting for features like building levels and area.
3. **Interactive Visualization Map:**
   An interactive Folium-based map visualizing prioritized infrastructure, color-coded by urgency, to support quick situational awareness for disaster response teams.
4. **Documentation and Scripts:**
   Well-documented Python scripts for data acquisition, preprocessing, scoring, and visualization, along with a user guide for reproducibility and potential reuse in other regions.
5. **Scalable Workflow:**
   A modular and adaptable workflow that can be applied to different cities or disaster scenarios by updating input geospatial and damage data.

Progress Overview

Based on the information gathered so far, the project is progressing steadily, with the core architectural design and key functionalities implemented. The primary tasks related to system architecture, component specification, and communication between the Flask and Tkinter applications are complete. The route calculation logic and initial GUI development are also in place.

Completed:

- System Architecture Design
- Component Specification
- Communication Implementation
- Route Calculation Logic
- Initial GUI Development

Behind Schedule:

- GUI Refinement
- Performance Optimization
- Error Handling and Robustness
- Testing and Validation
- Deployment and Documentation

The project is neither ahead nor behind schedule overall.

Codebase Information

Repository Link: https://github.com/gunottam/disaster_recovery_using_AI

Branch: Main branch

Important Commits:

1. Added code to download building and infrastructure data from OSM
2. Loaded NASA and UNOSAT raster files and handled missing values
3. Implemented damage score normalization and combined with infrastructure data
4. Assigned importance weights based on infrastructure type and building levels
5. Created interactive map to visualize damage severity across Kathmandu

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
1. Raster file loading	1. Passed	1. Data loaded and verified successfully.
2. OSM infrastructure data extraction	2. Passed	2. OSM data filtered correctly by tags.
3. Damage normalization	3. Passed	3. Damage normalization applied accurately.
4. Spatial join of raster and vector data	4. Passed with minor spatial mismatches	4. Minor discrepancies due to projection differences.
5. Composite scoring and ranking	5. Passed	5. Visualizations rendered as expected.

Deliverables Progress

All key data acquisition tasks, including collecting spatial data from OpenStreetMap and damage assessment data from NASA and UNOSAT, have been successfully completed. The preprocessing of this data, involving filtering and cleaning, is also finished. We have normalized and integrated the damage scores and assigned importance weights to different infrastructure types based on their criticality. The composite scoring and ranking of infrastructure elements have been completed as well. The interactive visualization map is currently in progress, with ongoing refinements to improve usability and clarity. Finally, the preparation of the project report and presentation is still pending and will be completed soon.